

SISTEMAS ELECTRÓNICOS DIGITALES

CURSO 2025 / 2026

TRABAJO GRUPAL: IMPLEMENTACIÓN DE JUEGO DEL TETRIS EN VHDL

APELLIDOS, NOMBRE: Arriero Serrano, Felipe

APELLIDOS, NOMBRE: Morcillo García, Gonzalo

APELLIDOS, NOMBRE: Yankowich Oliver, Iván

GRUPO: EE-403

FECHA MÁXIMA DE ENTREGA: 21/01/2026

ÍNDICE

1. Descripción General del Proyecto	4
2. Diagrama de Bloques.....	5
2.1. Diagrama de Bloques por Funcionalidad.....	5
2.2. Diagrama de Bloques RTL	6
2.3. Esquema de Diseño Elaborado	8
3. Entidades del Proyecto	10
3.1. Generación de Señales de Temporización del Juego (<i>clk_divider</i>)	10
3.2. Generación y Gestión de Reloj (<i>clk_wiz_tetris</i>)	11
3.3. Gestión de Entradas del Usuario (<i>input_controller</i>).....	11
3.4. Definición Geométrica de las Piezas (<i>piece_rom</i>).....	12
3.5. Generación Pseudoaleatoria de Piezas (<i>lfsr_random</i>)	13
3.6. Control de la Pieza Activa (<i>current_piece_controller</i>)	14
3.7. Detector de Colisiones (<i>collision_detector</i>).....	15
3.8. Gestión de Tablero de Juego (<i>board_memory</i>).....	16
3.9. Control Global del Juego (<i>game_controller</i>).....	18
3.10. Renderizado Gráfico del Juego (<i>render_unit</i>)	19
3.11. Generación de la Señal VGA (<i>vga_controller</i>)	21
3.12. Gestión de la Puntuación (<i>score_counter</i>).....	22
3.13. Visualización en Display de Siete Segmentos(<i>seven_seg_driver</i>)	23
3.14. Integración del Sistema (<i>tetris_top</i>).....	23
4. Entradas y Salidas.....	26
4.1. Entradas.....	26
4.2. Salidas	26

5. Funcionamiento del Juego	27
5.1. Inicio de la Partida	27
5.2. Control y Movimiento de las Piezas	27
5.3. Fijación de Piezas y Eliminación de Líneas	28
5.4. Sistema de Puntuación y Niveles	28
5.5. Condiciones de Victoria y Derrota	29
5.6. Reinicio de la Partida	30
6. Testbenches	31
7. Distribución del Trabajo en Grupo y Uso de GitHub.....	32
8. Funcionalidades Implementadas.....	33
9. Problemas y Soluciones.....	35
9.1. Problema de Conexión y Visualización en Pantalla por VGA	35
9.2. Problema de Colisiones y Coherencia Temporal por Frecuencias de Reloj	35
9.3. Problema de Reset y Arranque No Determinista del Sistema	35
9.4. Problema de Reinicio de Partida sin Reset Físico	36
9.5. Problema en la Gestión de la Puntuación y Borrado de Líneas	36
9.6. Problema de Bloqueo de la Pieza al Tocar el Suelo	36
9.7. Problemas de Integración y Organización del Proyecto en Vivado	37
10. Conclusiones.....	38

1. Descripción General del Proyecto

El presente proyecto consiste en el diseño e implementación de un videojuego tipo Tetris utilizando VHDL, destinado a su ejecución sobre plataformas FPGA, concretamente en las placas Nexys4 DDR y Basys3, dentro del contexto de la asignatura Sistemas Electrónicos Digitales del Grado en Ingeniería impartido en la ETSIDI (UPM).

El objetivo principal del trabajo es aplicar de forma práctica los conceptos de diseño digital síncrono, descripción hardware mediante VHDL y arquitectura modular, desarrollando un sistema funcional y verificable que interactúe con el usuario mediante los recursos hardware disponibles en la FPGA. El proyecto ha sido concebido con un enfoque plenamente académico, priorizando la claridad del diseño, la correcta estructuración en entidades y el uso eficiente de los recursos lógicos.

El juego se visualiza a través de una salida VGA, generada íntegramente desde la FPGA, y se controla exclusivamente mediante los pulsadores físicos integrados en la placa, sin recurrir a periféricos externos. El sistema implementa la lógica completa del juego Tetris, incluyendo el movimiento y rotación de piezas, la detección de colisiones, la eliminación de filas completas, un sistema de puntuación dinámico y la gestión de distintos estados de funcionamiento mediante una máquina de estados finitos.

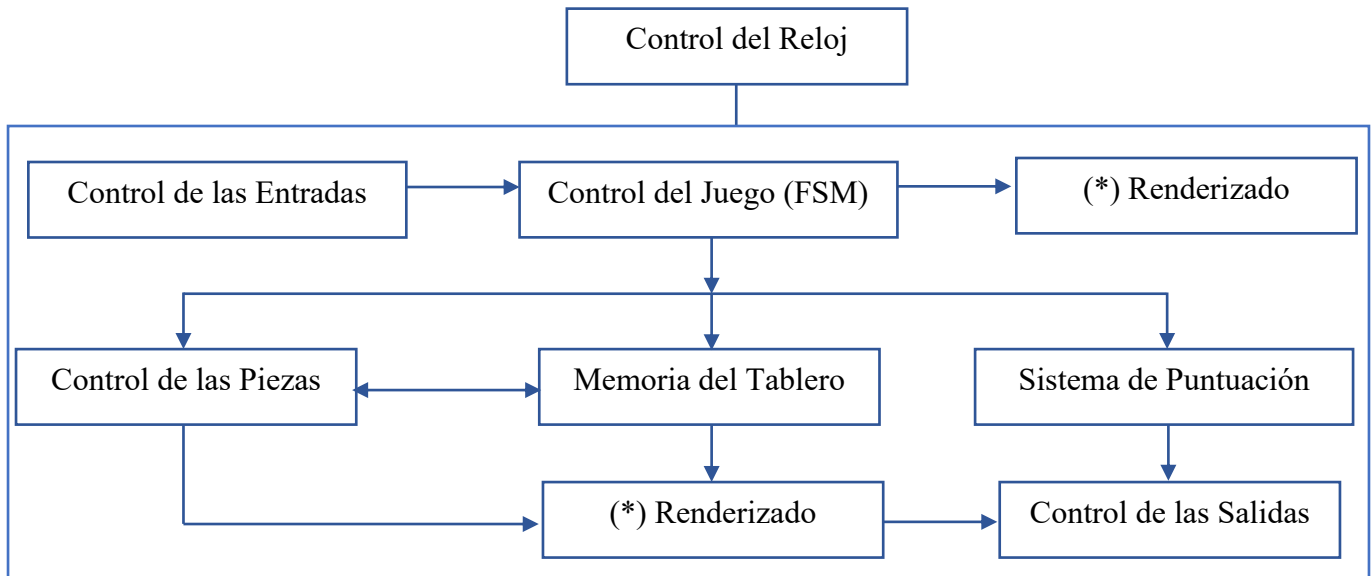
Adicionalmente, el diseño contempla diferentes pantallas de estado, como juego en curso, *Game Over* y *Win*, permitiendo reiniciar una partida desde los estados finales sin necesidad de reiniciar físicamente la placa. La puntuación se muestra en tiempo real a través del display de 7 segmentos, y la dificultad del juego se incrementa progresivamente mediante el aumento de la velocidad de caída de las piezas en función de la puntuación acumulada.

Desde el punto de vista del diseño hardware, el proyecto se ha desarrollado siguiendo criterios de modularidad, dividiendo el sistema en múltiples entidades VHDL bien definidas y reutilizables, haciendo uso de genéricos para evitar la duplicación de código y facilitar la escalabilidad. Asimismo, se ha llevado a cabo un proceso de verificación funcional mediante testbenches, asegurando el correcto comportamiento de los distintos bloques del sistema antes de su integración final.

2. Diagrama de Bloques

2.1. Diagrama de Bloques por Funcionalidad

El diagrama de bloques por funcionalidad se incluye con el objetivo de ofrecer una visión general del sistema y facilitar la comprensión de las principales funciones implementadas y su relación entre sí. En este diagrama se representan los distintos subsistemas del juego y los flujos de información entre ellos, sin entrar en el detalle de la implementación a nivel de señales o procesos internos.



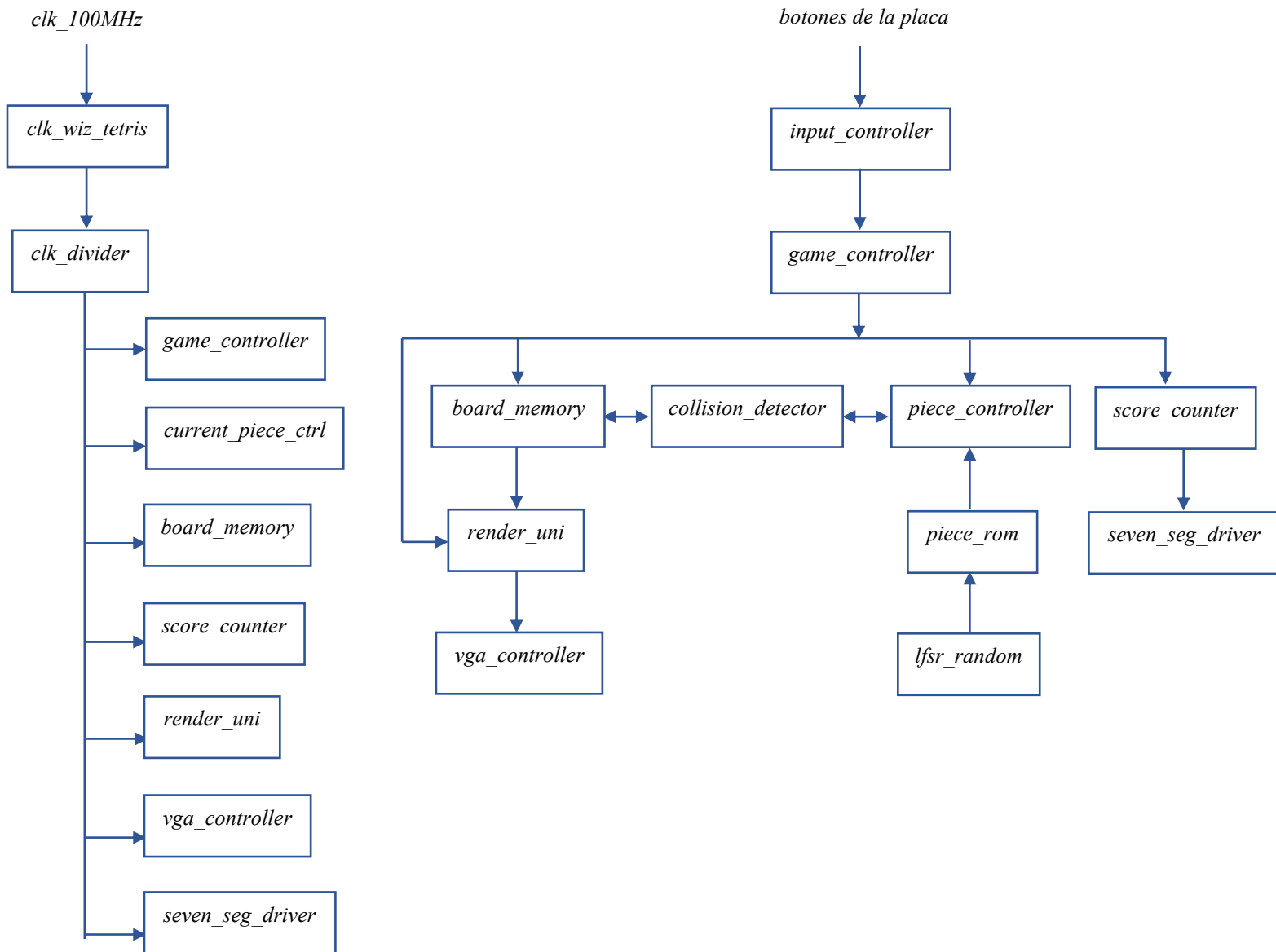
El sistema se organiza en torno a un bloque central de control del juego, implementado mediante una máquina de estados finitos, que gestiona el flujo general del juego y coordina el resto de los subsistemas.

El control de las entradas se encarga de procesar las señales procedentes de los pulsadores de la placa y generar las órdenes de movimiento y control del juego. El control de las piezas y la memoria del tablero gestionan, respectivamente, la pieza activa y el estado del tablero de juego, incluyendo la detección de colisiones y la fijación de piezas.

El sistema de puntuación calcula la puntuación obtenida durante la partida y envía la información al bloque de control de las salidas, encargado de mostrarla mediante el display de siete segmentos. Finalmente, el bloque de renderizado genera la información gráfica que se envía al sistema de salida VGA para su visualización en pantalla.

2.2. Diagrama de Bloques RTL

El diagrama de bloques RTL presenta una representación estructural de alto nivel del sistema, basada en las entidades VHDL que componen el diseño. Este diagrama permite identificar los principales módulos implementados y sus interconexiones, manteniendo un nivel de abstracción adecuado para facilitar su comprensión.



El diagrama de bloques RTL presenta una visión estructural de alto nivel del sistema desarrollado, basada en las entidades VHDL realmente instanciadas en la entidad superior del diseño (*tetris_top*). A diferencia del diagrama de bloques por funcionalidad, este esquema se centra en cómo está construido el sistema, mostrando los principales módulos hardware y las relaciones existentes entre ellos, manteniendo un nivel de abstracción que facilita su comprensión sin entrar en el detalle de señales internas o lógica combinacional auxiliar.

La entidad superior (*tetris_top*) actúa como elemento de integración del sistema, siendo responsable de interconectar los distintos módulos, distribuir las señales de reloj y reset, y definir la interfaz con los periféricos externos de la placa FPGA, tales como los pulsadores, la salida VGA y el display de siete segmentos. En este nivel no se implementa lógica funcional compleja, sino que se organiza la estructura general del diseño.

El sistema se basa en un funcionamiento completamente síncrono, para lo cual se dispone de un bloque de gestión del reloj, compuesto por los módulos encargados de generar y adaptar las señales de temporización necesarias. A partir del reloj principal de la placa, estos bloques proporcionan las señales de reloj y habilitación utilizadas por el resto de los módulos del sistema, garantizando un comportamiento coordinado de la lógica de control, procesamiento y visualización del juego.

Las entradas del sistema, procedentes de los pulsadores físicos de la placa, son gestionadas por el bloque *input_controller*. Este módulo se encarga de sincronizar las señales de entrada con el reloj del sistema y de generar señales de control adecuadas para el resto del diseño. De este modo, se evita la propagación de señales asíncronas y se garantiza un funcionamiento estable y predecible del sistema.

El núcleo del diseño está constituido por el bloque *game_controller*, implementado como una máquina de estados finitos (FSM). Este módulo gobierna el flujo general del juego y determina el estado global del sistema en cada instante, como por ejemplo el estado de juego en curso, fin de partida (Game Over) o victoria (Win). A partir de este estado, el *game_controller* coordina el funcionamiento del resto de módulos, habilitando o bloqueando determinadas acciones y sincronizando las distintas etapas del juego.

La gestión de la pieza activa se realiza mediante el bloque *current_piece_ctrl*, encargado de controlar la posición y rotación de la pieza que se encuentra en movimiento. Este módulo recibe las órdenes de control procedentes del *game_controller* y de las entradas del usuario, y calcula la evolución de la pieza dentro del tablero. Para ello, interactúa con otros bloques auxiliares, como los encargados de la generación de piezas y la detección de colisiones, los cuales están instanciados en la entidad superior y proporcionan soporte a la lógica principal.

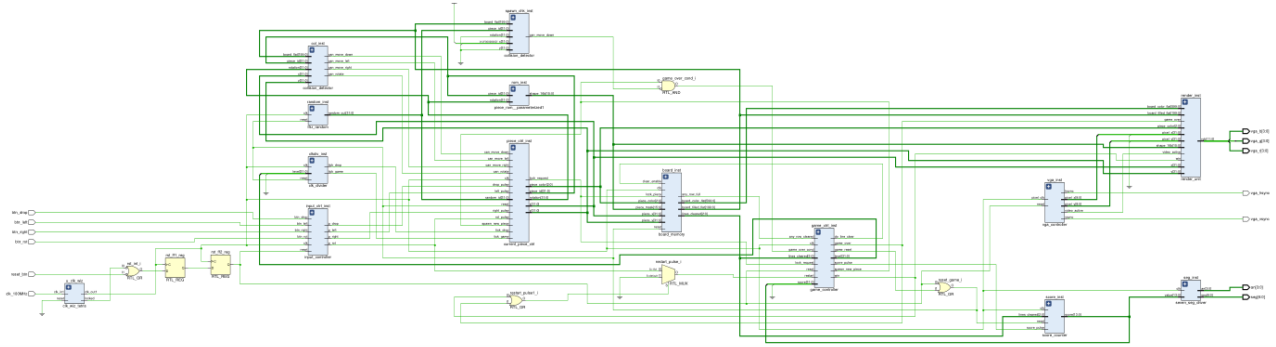
El estado del tablero de juego se mantiene en el bloque *board_memory*, que actúa como una memoria donde se almacena la ocupación de cada celda del tablero. Este módulo recibe información del controlador de la pieza activa para fijar las piezas cuando estas colisionan o alcanzan su posición final, y se encarga también de detectar y eliminar las filas completas. La interacción entre *current_piece_ctrl* y *board_memory* es fundamental para asegurar la coherencia del estado del juego y la correcta detección de colisiones.

El sistema de puntuación se implementa mediante el bloque *score_counter*, que calcula la puntuación obtenida en función de las acciones realizadas durante el juego, como la eliminación de filas. Este módulo recibe señales de control procedentes del *game_controller* y del estado del tablero, y genera el valor de puntuación que se muestra al usuario. La visualización de dicha puntuación se realiza a través del bloque *seven_seg_driver*, encargado de controlar el *display* de siete segmentos integrado en la placa FPGA.

La visualización gráfica del juego se gestiona en dos etapas claramente diferenciadas. En primer lugar, el bloque *render_unit* combina la información procedente de la memoria del tablero, del controlador de la pieza activa y del estado global del juego proporcionado por el *game_controller*. A partir de estos datos, genera una representación lógica de la escena que debe mostrarse en pantalla, incluyendo el tablero, la pieza en movimiento y las pantallas finales de estado. En una segunda etapa, el bloque *vga_controller* se encarga de transformar dicha información en las señales necesarias para la salida VGA. Este módulo genera las señales de sincronismo horizontal y vertical, así como las componentes de color correspondientes a cada píxel, permitiendo la correcta visualización del juego en un monitor externo compatible.

2.3. Esquema de Diseño Elaborado

Como complemento a los diagramas de bloques presentados, se incluye a continuación el *schematic* generado automáticamente por la herramienta Vivado a partir del diseño descrito en VHDL. Este esquema refleja la jerarquía real del sistema y la interconexión entre las distintas entidades instanciadas en la entidad superior, permitiendo contrastar la arquitectura conceptual y estructural mostrada en los diagramas de bloques con la implementación efectiva del diseño. Dado su alto nivel de detalle, el *schematic* se aporta como apoyo visual y evidencia del diseño, no siendo su objetivo facilitar la comprensión funcional del sistema, sino mostrar la correspondencia directa con el diseño RTL sintetizado.



3. Entidades del Proyecto

3.1. Generación de Señales de Temporización del Juego (*clk_divider*)

La entidad *clk_divider* se encarga de la generación de señales de temporización necesarias para el correcto funcionamiento del juego, a partir del reloj principal del sistema. Su función principal es proporcionar señales de tick que permiten sincronizar la lógica del juego y controlar la velocidad de caída de las piezas, adaptando dinámicamente esta última en función del nivel de dificultad.

Esta entidad se ha diseñado como un módulo parametrizable, haciendo uso de genéricos para facilitar el ajuste de los tiempos sin necesidad de modificar la arquitectura interna. En concreto, se definen parámetros para establecer la velocidad base del juego (*GAME_SPEED*), así como los valores asociados a la caída de las piezas (*DROP_BASE*, *DROP_STEP* y *DROP_MIN*), lo que permite adaptar el comportamiento temporal del sistema de forma flexible.

La entidad dispone como entradas del reloj del sistema (*clk*) y de una señal de reset síncrono, además de una señal *level* que indica el nivel actual del juego. A partir de estas señales, se generan dos salidas principales: *tick_game*, que marca el ritmo general del juego, y *tick_drop*, que determina los instantes en los que la pieza activa debe descender automáticamente.

El diseño interno se basa en el uso de contadores síncronos, implementados mediante señales de tipo *unsigned*. Para la generación del tick del juego (*tick_game*), se emplea un contador que se incrementa con cada flanco de subida del reloj y se reinicia cuando alcanza el valor definido por el genérico *GAME_SPEED*, generando un pulso de activación de un ciclo de duración.

Por otro lado, el tick asociado a la caída de las piezas (*tick_drop*) se genera a partir de un segundo contador cuyo periodo se ajusta dinámicamente en función del nivel del juego. Para ello, se calcula un periodo de caída (*drop_period*) que disminuye progresivamente a medida que aumenta el nivel, incrementando así la dificultad. Este cálculo incluye mecanismos de saturación que garantizan que el periodo de caída se mantenga dentro de unos límites razonables, evitando valores excesivamente pequeños que puedan comprometer el correcto funcionamiento del sistema.

Gracias a esta estructura, la entidad *clk_divider* permite desacoplar la temporización del juego del reloj principal, facilitando el diseño de la lógica de control y asegurando un comportamiento temporal coherente y escalable. Su carácter modular y parametrizable contribuye a la claridad del diseño y a la reutilización del código en posibles ampliaciones del sistema.

3.2. Generación y Gestión de Reloj (*clk_wiz_tetris*)

El bloque *clk_wiz_tetris* corresponde a un IP generado automáticamente por Vivado (*Clocking Wizard*) cuya función es proporcionar las señales de reloj necesarias para el sistema a partir del reloj principal de la placa FPGA. En este proyecto, el reloj de entrada (*clk_in1*) procede del oscilador de la placa (100 MHz), y el bloque genera como salida un reloj adaptado a los requisitos de la interfaz VGA y del resto del sistema.

En particular, el diseño requiere un reloj de píxel de 25 MHz para el modo VGA 640×480@60 Hz, por lo que el *Clock Wizard* se utiliza para dividir y acondicionar el reloj de 100 MHz mediante recursos de reloj dedicados de la FPGA (MMCM/PLL), garantizando estabilidad temporal y bajo jitter en la señal generada. Esta solución resulta preferible frente a divisores lógicos simples cuando se requiere un reloj estable para vídeo.

Además, el IP proporciona una señal *locked*, que indica que el generador de reloj ha alcanzado un estado estable y que la salida es válida. En la entidad superior, dicha señal se utiliza como parte del mecanismo de reset robusto, evitando que el sistema comience a operar antes de que el reloj de trabajo esté correctamente estabilizado. De este modo se mejora la fiabilidad del arranque y se previenen comportamientos indeterminados durante los primeros instantes tras la configuración de la FPGA.

En conjunto, *clk_wiz_tetris* actúa como un bloque de infraestructura, esencial para adaptar el reloj del sistema a los requisitos temporales del proyecto y asegurar una sincronización adecuada de todos los módulos, especialmente los relacionados con la salida VGA.

3.3. Gestión de Entradas del Usuario (*input_controller*)

La entidad *input_controller* se encarga de la gestión y acondicionamiento de las entradas físicas del sistema, concretamente de los pulsadores utilizados para el control del juego. Su función principal es garantizar que las señales procedentes de los botones sean estables, síncronas y libres de rebotes, proporcionando al resto del sistema señales de control limpias y fiables.

Esta entidad se ha diseñado como un módulo parametrizable, incorporando genéricos que permiten adaptar su funcionamiento a distintas frecuencias de reloj y tiempos de antirrebote sin modificar el código interno. En particular, el genérico *CLK_FREQ_HZ* define la frecuencia del reloj del sistema, mientras que *DEBOUNCE_MS* establece el tiempo mínimo durante el cual una señal debe permanecer estable para ser considerada válida. Este planteamiento mejora la reutilización del módulo y facilita su adaptación a diferentes plataformas FPGA.

Las entradas del módulo corresponden al reloj y al reset del sistema, junto con las señales de los pulsadores físicos asociados al movimiento lateral, rotación y caída acelerada de la pieza. Como salidas, el bloque genera señales de pulso (*p_left*, *p_right*, *p_rot* y *p_drop*) de duración de un solo ciclo de reloj, que indican la detección de una pulsación válida. Este formato de salida simplifica la integración con la lógica de control del juego, evitando la necesidad de gestionar pulsaciones prolongadas.

El diseño interno implementa un mecanismo de antirrebote completamente síncrono, basado en la combinación de sincronización mediante biestables y contadores temporales. Para cada botón se emplea una estructura idéntica, reutilizada mediante un tipo de datos compuesto (*record*), lo que permite mantener un diseño compacto, legible y fácil de mantener. Esta estructura almacena el estado sincronizado de la entrada, el valor estable detectado y un contador que verifica que el cambio de estado se mantiene durante el tiempo de antirrebote configurado.

Cada pulsador es procesado de forma independiente dentro de un único proceso secuencial, garantizando que todas las entradas se evalúan de manera coherente en cada flanco de subida del reloj. Una vez que una señal se considera estable, se detecta el flanco de subida comparando el estado actual con el estado estable previo, generando así un pulso de activación de un ciclo. De este modo, se evita que una pulsación prolongada produzca múltiples acciones no deseadas.

Gracias a esta implementación, el bloque *input_controller* actúa como una interfaz robusta entre el mundo físico y la lógica digital, aislando al resto del sistema de los efectos propios de los pulsadores mecánicos. Su diseño modular y parametrizable contribuye a la claridad general del proyecto y permite un control preciso y fiable del juego por parte del usuario.

3.4. Definición Geométrica de las Piezas (*piece_rom*)

La entidad *piece_rom* implementa una memoria de solo lectura (ROM) combinacional que contiene la definición geométrica de todas las piezas del juego Tetris, así como sus distintas rotaciones. Su función principal es proporcionar, de forma inmediata y sin necesidad de lógica secuencial, la forma correspondiente a una pieza concreta en una orientación determinada.

Esta entidad recibe como entradas un identificador de pieza (*piece_id*) y un identificador de rotación (*rotation*), y genera como salida un vector de 16 bits (*shape_16b*) que representa la geometría de la pieza seleccionada. Cada pieza se codifica como una matriz de 4×4, donde cada bit indica la presencia o ausencia de un bloque en una celda concreta. Esta representación compacta permite un manejo sencillo y eficiente de las piezas dentro del resto de la lógica del sistema.

Internamente, la entidad define un conjunto de constantes estructuradas, organizadas mediante tipos de datos compuestos (*array*), que almacenan las siete piezas clásicas del Tetris y sus cuatro posibles rotaciones. Cada pieza se asocia a un conjunto fijo de patrones binarios, lo que elimina la necesidad de cálculos dinámicos para obtener las rotaciones y simplifica significativamente el diseño del sistema.

El acceso a la información se realiza de forma puramente combinacional, seleccionando directamente el patrón correspondiente en función de las entradas. De este modo, el módulo introduce una latencia nula y puede ser utilizado de manera transparente por otros bloques, como el controlador de la pieza activa o los módulos de detección de colisiones, sin afectar a la temporización global del sistema.

Desde el punto de vista del diseño, la entidad *piece_rom* contribuye a una separación clara de responsabilidades, aislando la definición estática de las piezas del resto de la lógica de control del juego. Esta modularización mejora la legibilidad del código, facilita su mantenimiento y permite posibles ampliaciones futuras, como la modificación o incorporación de nuevas piezas, sin necesidad de alterar otros módulos del sistema.

3.5. Generación Pseudoaleatoria de Piezas (*lfsr_random*)

La entidad *lfsr_random* se encarga de la generación de valores pseudoaleatorios utilizados para seleccionar el tipo de pieza que aparece en el juego. Su función principal es proporcionar una secuencia de números que permita variar la aparición de las piezas, evitando patrones deterministas y aportando mayor dinamismo al desarrollo de la partida.

El módulo recibe como entradas el reloj del sistema y una señal de reset, y genera como salida un valor entero (*random_out*) comprendido entre 0 y 6, correspondiente a los siete tipos de tetrominós disponibles en el juego. Esta salida es empleada por los módulos de control de la pieza activa para seleccionar la geometría adecuada desde la memoria de piezas.

El diseño interno se basa en un registro de desplazamiento con realimentación lineal (LFSR) de 8 bits, implementado de forma completamente síncrona. En cada flanco de subida del reloj, el contenido del registro se desplaza y se introduce un nuevo bit calculado mediante una combinación *XOR* de varios bits del propio registro, siguiendo un polinomio de realimentación estándar. Este tipo de estructura permite generar secuencias pseudoaleatorias de manera eficiente, utilizando un número reducido de recursos hardware.

La inicialización del registro se realiza mediante una semilla fija al activarse la señal de reset, lo que garantiza un estado inicial conocido del sistema. A partir de dicha semilla, la evolución del LFSR

produce una secuencia repetitiva de longitud elevada, suficiente para el contexto del juego y adecuada para la selección de piezas sin necesidad de mecanismos aleatorios complejos.

Para adaptar la salida del LFSR al rango requerido por el sistema, se emplean los bits menos significativos del registro, que se convierten a un valor entero y se reducen mediante una operación módulo 7. De este modo, se garantiza que el valor generado corresponda siempre a uno de los tipos de pieza definidos, manteniendo la simplicidad del diseño.

Desde el punto de vista del sistema global, la entidad *lfsr_random* constituye un bloque auxiliar que aporta variabilidad al juego sin afectar al control principal ni a la temporización del sistema. Su implementación modular y su bajo coste en recursos contribuyen a un diseño equilibrado, fácilmente integrable y coherente con el resto de la arquitectura desarrollada.

3.6. Control de la Pieza Activa (*current_piece_controller*)

La entidad *current_piece_ctrl* es la encargada de la gestión completa de la pieza activa del juego, constituyendo uno de los núcleos funcionales del sistema. Su función principal es mantener y actualizar el estado de la pieza que se encuentra en movimiento, incluyendo su posición, orientación, identificación y color, así como coordinar su interacción con el tablero y el controlador general del juego.

Este módulo ha sido diseñado de forma parametrizable, incorporando genéricos que definen las dimensiones del tablero (*BOARD_WIDTH* y *BOARD_HEIGHT*). Gracias a ello, el controlador de la pieza no depende de valores fijos, lo que mejora la reutilización del código y facilita posibles modificaciones futuras del tamaño del área de juego sin alterar la lógica interna.

Las entradas del bloque incluyen, además del reloj y la señal de reset, los pulsos de control generados por el módulo de entradas (desplazamiento lateral, rotación y caída manual), así como las señales de temporización (*tick_game* y *tick_drop*) que regulan el avance automático del juego. Asimismo, el módulo recibe señales de habilitación (*can_move_left*, *can_move_right*, *can_rotate* y *can_move_down*) que indican si una determinada acción es válida en cada instante, normalmente calculadas por los módulos de detección de colisiones y gestión del tablero.

La entidad también recibe una señal de control (*spawn_new_piece*) procedente del *game_controller*, junto con un identificador pseudoaleatorio (*random_id*) que determina el tipo de la nueva pieza a generar. Cuando se solicita la aparición de una nueva pieza, el módulo inicializa su estado asignando una posición de aparición predefinida, una orientación inicial y un color asociado a la pieza, garantizando un estado coherente desde el primer ciclo de funcionamiento.

Internamente, el estado de la pieza activa se almacena en registros síncronos, que mantienen la posición horizontal (x), la posición vertical (y), la rotación, el identificador de la pieza y su color. Todas las actualizaciones se realizan en un único proceso secuencial, sincronizado con el flanco de subida del reloj, lo que asegura un comportamiento determinista y fácil de analizar.

El bloque implementa la lógica de movimiento de la pieza, permitiendo desplazamientos laterales y rotaciones únicamente cuando las señales de habilitación lo permiten. De este modo, se evita que la pieza atraviese los límites del tablero o colisione con otras piezas ya fijadas. La caída de la pieza se produce de manera automática mediante la señal *tick_drop*, o de forma manual cuando el usuario acciona el pulsador correspondiente, manteniendo siempre la coherencia con las restricciones impuestas por el tablero.

Un aspecto destacable del diseño es la gestión del bloqueo de la pieza cuando esta alcanza el fondo del tablero o entra en colisión con otras piezas. En este caso, el módulo genera una señal de petición de bloqueo (*lock_request*), que indica al sistema que la pieza debe fijarse definitivamente en el tablero. A partir de ese momento, la pieza entra en un estado congelado, en el que no se permiten más movimientos, hasta que el controlador del juego solicita explícitamente la generación de una nueva pieza.

Gracias a esta arquitectura, la entidad *current_piece_ctrl* actúa como un módulo autónomo y bien delimitado, responsable exclusivamente del comportamiento de la pieza activa, mientras que la toma de decisiones globales queda delegada en el *game_controller*. Esta separación de responsabilidades contribuye a una arquitectura clara, modular y fácil de mantener, facilitando tanto la depuración como la verificación individual del sistema.

3.7. Detector de Colisiones (*collision_detector*)

La entidad *collision_detector* se encarga de la detección de colisiones entre la pieza activa y el tablero de juego, constituyendo un bloque fundamental para garantizar la coherencia del movimiento y la correcta aplicación de las reglas del Tetris. Su función principal es evaluar si una determinada acción sobre la pieza (desplazamiento lateral, caída o rotación) es válida en función del estado actual del tablero y de los límites del área de juego.

Este módulo se ha diseñado de forma parametrizable, incorporando genéricos que definen las dimensiones del tablero (*BOARD_WIDTH* y *BOARD_HEIGHT*). Gracias a esta parametrización, el detector de colisiones es independiente del tamaño concreto del tablero y puede reutilizarse fácilmente en configuraciones distintas sin modificar su arquitectura interna.

Las entradas del bloque incluyen la información completa del estado de la pieza activa, concretamente su identificador (*piece_id*), su rotación actual (*rotation*) y su posición dentro del tablero (x e y), así como una representación plana del tablero (*board_flat*) que indica qué celdas se encuentran ocupadas. A partir de estos datos, el módulo genera como salidas una serie de señales lógicas (*can_move_left*, *can_move_right*, *can_move_down* y *can_rotate*) que indican si cada una de las posibles acciones es permitida en el instante actual.

Internamente, la entidad utiliza la memoria de piezas (*piece_rom*) para obtener la forma actual de la pieza y su forma tras una posible rotación. Ambas representaciones se codifican como vectores de 16 bits que describen una matriz de 4×4, lo que permite evaluar de manera sistemática la ocupación de cada bloque que compone la pieza.

El núcleo del bloque se basa en una función combinacional de comprobación de colisiones, que analiza cada uno de los bloques activos de la pieza y calcula su posición absoluta dentro del tablero. Para cada bloque se verifica, por un lado, que no se excedan los límites del área de juego y, por otro, que la celda correspondiente del tablero no esté ya ocupada. En caso de detectarse cualquiera de estas situaciones, la función indica la existencia de una colisión.

A partir de esta función, el módulo evalúa de forma independiente las distintas acciones posibles sobre la pieza. Para ello, se simulan de manera combinacional los desplazamientos laterales, la caída vertical y la rotación, comprobando en cada caso si la nueva posición resultante produciría una colisión. Las salidas del bloque reflejan directamente el resultado de estas comprobaciones, permitiendo al controlador de la pieza tomar decisiones informadas sin necesidad de realizar cálculos adicionales.

Desde el punto de vista arquitectónico, la entidad *collision_detector* actúa como un bloque auxiliar puramente combinacional, sin elementos de memoria ni dependencia del reloj. Esta característica permite que sus salidas respondan de manera inmediata a los cambios en el estado de la pieza o del tablero, facilitando un diseño limpio y eficiente del sistema de control. Además, al separar la detección de colisiones de la lógica de movimiento, se refuerza la modularidad del diseño y se simplifica la verificación y depuración del sistema completo.

3.8. Gestión de Tablero de Juego (*board_memory*)

La entidad *board_memory* es la encargada de almacenar y gestionar el estado completo del tablero de juego, actuando como la memoria principal del sistema Tetris. Su función principal es mantener la información relativa a qué celdas del tablero están ocupadas, el color asociado a cada una de ellas, así como realizar la fijación de piezas y la detección y eliminación de filas completas.

Este módulo se ha diseñado de forma parametrizable, incorporando genéricos que definen las dimensiones del tablero (*BOARD_WIDTH* y *BOARD_HEIGHT*). Gracias a esta parametrización, el diseño es independiente del tamaño concreto del tablero y puede adaptarse fácilmente a otras configuraciones sin modificar la lógica interna.

Las entradas del bloque incluyen el reloj y la señal de reset del sistema, así como señales de control procedentes del controlador del juego. En particular, la señal *lock_piece* indica que la pieza activa debe fijarse definitivamente en el tablero, mientras que *clear_enable* habilita el proceso de eliminación de filas completas. Adicionalmente, el módulo recibe la posición de la pieza (*piece_x*, *piece_y*), su máscara de forma (*piece_mask*) y su color (*piece_color*), información necesaria para actualizar correctamente el estado del tablero.

Internamente, el tablero se representa mediante una estructura bidimensional de registros, donde cada celda almacena dos campos: un indicador de ocupación (*filled*) y el color correspondiente. Esta representación permite un acceso claro y directo a cada posición del tablero, facilitando tanto la fijación de piezas como la detección de filas completas.

Cuando se solicita la fijación de una pieza, el módulo recorre la máscara de la pieza activa, calcula la posición absoluta de cada uno de sus bloques y actualiza las celdas correspondientes del tablero, siempre que se encuentren dentro de los límites definidos. De este modo, la pieza pasa a formar parte permanente del tablero una vez ha alcanzado su posición final.

Tras la actualización del tablero, el bloque evalúa de manera secuencial cada fila para determinar si está completamente ocupada. En caso de detectarse una o varias filas completas, se genera una señal indicativa (*any_row_full*) y se contabiliza el número de líneas a eliminar. Este valor se almacena en un registro interno y se proporciona como salida (*lines_cleared*), permitiendo que otros módulos del sistema, como el controlador de puntuación, actúen en consecuencia.

Cuando la eliminación de filas está habilitada, el módulo reconstruye el tablero desplazando hacia abajo las filas no completas y limpiando las filas superiores, reproduciendo el comportamiento clásico del juego Tetris. Este proceso se realiza de forma controlada dentro de un único ciclo de reloj, garantizando la coherencia del estado del tablero en todo momento.

Como salidas, la entidad *board_memory* proporciona una representación aplanada del tablero, tanto del estado de ocupación como de los colores de cada celda. Estas salidas están destinadas a ser utilizadas por otros bloques del sistema, como el detector de colisiones y el módulo de renderizado, permitiendo una integración sencilla con el resto de la arquitectura.

En conjunto, *board_memory* constituye un bloque central de gestión de datos, responsable de mantener la consistencia del estado del juego. Su diseño modular, parametrizable y claramente estructurado contribuye de forma decisiva a la claridad, robustez y mantenibilidad del sistema completo.

3.9. Control Global del Juego (*game_controller*)

La entidad *game_controller* implementa la lógica de control global del juego, actuando como el elemento central de coordinación entre los distintos módulos del sistema. Su función principal es gobernar el flujo de ejecución del juego, gestionar los cambios de estado, controlar la generación de nuevas piezas, la eliminación de líneas, la progresión de niveles y la detección de las condiciones de victoria y derrota.

Este bloque se ha diseñado como una máquina de estados finitos (FSM), lo que permite estructurar de forma clara y determinista el comportamiento del juego. Mediante esta aproximación, cada fase del juego se representa mediante un estado bien definido, facilitando la comprensión del diseño, su verificación y su depuración.

Las entradas del módulo incluyen señales procedentes de los bloques de control de la pieza activa y de la memoria del tablero, como la petición de bloqueo de una pieza (*lock_request*) y la indicación de que existen filas completas (*any_row_cleared*), así como el número de líneas eliminadas (*lines_cleared*). Adicionalmente, el controlador recibe información del estado global del juego, como la puntuación actual (*score*) y una señal que indica la imposibilidad de generar una nueva pieza (*game_over_cond*). También se incluye una entrada de reinicio lógico (*restart*) que permite comenzar una nueva partida sin necesidad de reiniciar físicamente la placa.

El comportamiento del juego se articula a través de una serie de estados bien diferenciados, que representan las distintas fases del ciclo de juego: generación de una nueva pieza, juego activo, bloqueo de la pieza, comprobación y eliminación de líneas, aumento de nivel, y estados finales de *Game Over* y *Win*. Las transiciones entre estos estados dependen de las señales de entrada y de las condiciones internas del sistema, garantizando una evolución coherente del juego.

En el estado de juego activo, el controlador supervisa la aparición de condiciones especiales, como la fijación de una pieza o la consecución de la puntuación objetivo, que dan lugar a transiciones hacia estados de bloqueo o victoria, respectivamente. Tras el bloqueo de una pieza, el sistema evalúa si existen filas completas y, en caso afirmativo, activa el proceso de eliminación de líneas y la actualización de la puntuación.

La entidad también gestiona la progresión de niveles, manteniendo un contador interno de líneas eliminadas y actualizando el nivel del juego cuando se alcanza un umbral predefinido. Este nivel se proporciona como salida al resto del sistema, permitiendo adaptar dinámicamente la velocidad del juego y aumentar progresivamente la dificultad. Asimismo, se establece un nivel máximo para evitar valores fuera de rango.

Como salidas, el *game_controller* genera señales de control clave para el funcionamiento del sistema, tales como la orden de generar una nueva pieza (*spawn_new_piece*), la habilitación del borrado de líneas (*do_line_clear*) y un pulso de puntuación (*score_pulse*) sincronizado con la eliminación de filas. También se generan señales de estado global que indican si el juego ha terminado por derrota (*game_over*) o por victoria (*win*), así como una señal de reset lógico (*game_reset*) que permite reiniciar el estado interno del juego de forma controlada.

Finalmente, el módulo incluye una salida de depuración (*game_state*) que codifica el estado actual de la máquina de estados, facilitando la verificación y el análisis del comportamiento del sistema durante el desarrollo y las pruebas.

En conjunto, la entidad *game_controller* constituye el núcleo de decisión del sistema, coordinando de forma centralizada los distintos bloques del diseño y asegurando un comportamiento consistente, modular y fácilmente extensible del juego.

3.10. Renderizado Gráfico del Juego (*render_unit*)

La entidad *render_unit* es la encargada de la generación de la información gráfica del juego, transformando el estado lógico del sistema en una representación visual píxel a píxel que posteriormente será enviada al controlador VGA. Su función principal es determinar el color correspondiente a cada píxel de la pantalla en función del contenido del tablero, la pieza activa y el estado global del juego.

Este módulo se ha diseñado como un bloque puramente combinacional, sin dependencia directa del reloj, lo que permite que la salida gráfica se actualice de forma inmediata ante cualquier cambio en las entradas. De este modo, el renderizado del juego se mantiene coherente con el estado interno del sistema en todo momento.

Las entradas de la entidad incluyen, por un lado, la información completa del tablero de juego, tanto en lo relativo a las celdas ocupadas (*board_filled_flat*) como a los colores asociados a cada una de ellas (*board_color_flat*). Por otro lado, el módulo recibe el estado de la pieza activa, incluyendo su posición (x, y), su forma codificada en una máscara de 16 bits (*shape_16b*) y su color. Además, se

proporcionan señales de estado global (*game_over* y *win*) que permiten modificar el contenido visual mostrado en pantalla cuando la partida finaliza.

El *renderizador* recibe también la posición actual del píxel que se está generando (*pixel_x*, *pixel_y*) junto con una señal de habilitación de vídeo (*video_active*), procedentes del controlador VGA. A partir de esta información, el módulo calcula el color RGB correspondiente a cada píxel, expresado mediante un vector de 12 bits.

Desde el punto de vista geométrico, el módulo define una serie de parámetros constantes que determinan el tamaño de las celdas, las dimensiones del tablero en píxeles y su posición dentro de la pantalla. Gracias a estos parámetros, el tablero se representa centrado horizontalmente y con un tamaño uniforme, facilitando una visualización clara y proporcionada del juego.

Durante el funcionamiento normal del juego, el renderizador determina si el píxel actual pertenece al área del tablero, a su borde o al fondo de la pantalla. En el interior del tablero, se identifica la celda correspondiente y se decide el color final del píxel en función de una jerarquía bien definida: en primer lugar, se da prioridad a las celdas ya fijadas en el tablero; si estas no están ocupadas, se evalúa la presencia de la pieza activa en dicha posición. En caso contrario, se muestra el fondo del tablero. Este enfoque garantiza una representación visual coherente y evita conflictos entre la pieza en movimiento y las piezas ya fijadas.

Asimismo, el módulo incorpora la lógica necesaria para resaltar los bordes de las celdas, mejorando la percepción visual de la cuadrícula del juego. Para ello, se detectan los píxeles situados en los límites de cada celda y se les asigna un color específico, lo que contribuye a una mayor claridad gráfica.

En los estados finales del juego, el comportamiento del renderizador cambia para mostrar mensajes gráficos de *Game Over* o *Win*. Para ello, el módulo incluye una pequeña fuente de caracteres de 5×7 píxeles, escalable mediante un factor de ampliación, que permite dibujar texto directamente sobre la pantalla sin necesidad de recursos externos. Esta funcionalidad se implementa de manera combinacional, evaluando si el píxel actual pertenece a alguno de los caracteres que componen los mensajes finales.

En conjunto, la entidad *render_unit* constituye el vínculo entre la lógica interna del juego y su representación visual, concentrando toda la lógica de renderizado en un único bloque bien definido. Su diseño modular, parametrizable y completamente combinacional facilita su integración con el controlador VGA y contribuye de manera decisiva a la claridad y calidad visual del sistema completo.

3.11. Generación de la Señal VGA (*vga_controller*)

La entidad *vga_controller* se encarga de la generación de las señales de sincronismo y temporización necesarias para la salida VGA, así como del cálculo de las coordenadas del píxel actual que se está representando en pantalla. Su función principal es proporcionar una base temporal y espacial sobre la que el módulo de renderizado pueda generar la información gráfica del juego.

Este bloque opera de manera síncrona con un reloj de píxel de 25 MHz, obtenido a partir del reloj principal de la FPGA mediante un módulo de generación de reloj. Dicha frecuencia permite cumplir con el estándar VGA de 640×480 píxeles a 60 Hz, ampliamente soportado por monitores VGA convencionales.

Internamente, el controlador define un conjunto de parámetros temporales que especifican la duración de las distintas fases del barrido horizontal y vertical, incluyendo la zona visible, los intervalos de sincronismo y los periodos de porche delantero y trasero. Estos valores siguen las especificaciones del estándar VESA para el modo de vídeo utilizado.

El núcleo del módulo se basa en dos contadores síncronos, uno horizontal y otro vertical, que se incrementan en cada flanco de subida del reloj de píxel. El contador horizontal recorre todas las posiciones de una línea de vídeo y, al completarse, se reinicia y provoca el avance del contador vertical. Cuando ambos contadores alcanzan el final de sus respectivos periodos, se inicia un nuevo ciclo de refresco de la imagen.

A partir del valor de estos contadores, el módulo genera las señales de sincronismo horizontal (*hsync*) y vertical (*vsync*), activas en nivel bajo durante los intervalos correspondientes. Asimismo, se genera una señal *video_active* que indica si el píxel actual pertenece a la zona visible de la pantalla, permitiendo al módulo de renderizado inhibir la generación de color fuera de dicha región.

El controlador proporciona además las coordenadas del píxel visible actual (*pixel_x* y *pixel_y*), que representan la posición horizontal y vertical dentro de la zona visible de la pantalla. Estas coordenadas son utilizadas directamente por el bloque de renderizado para determinar el contenido gráfico que debe mostrarse en cada píxel.

Desde el punto de vista arquitectónico, la entidad *vga_controller* actúa como un bloque de infraestructura de vídeo, completamente independiente de la lógica del juego. Esta separación permite mantener un diseño modular y facilita la reutilización del controlador VGA en otros proyectos con requisitos gráficos similares.

En conjunto, este módulo establece el marco temporal y espacial necesario para la visualización del juego, garantizando una salida VGA estable, compatible y sincronizada con el resto del sistema.

3.12. Gestión de la Puntuación (*score_counter*)

La entidad *score_counter* se encarga de la gestión y actualización de la puntuación del juego, calculando los puntos obtenidos en función del número de líneas eliminadas en cada jugada. Su función principal es traducir los eventos relevantes del juego en una puntuación acumulada, siguiendo una regla de asignación de puntos coherente con el funcionamiento clásico del Tetris.

Las entradas del módulo incluyen el reloj y la señal de reset del sistema, así como una señal de control (*score_pulse*) que indica el instante en el que debe actualizarse la puntuación. Este pulso, de duración de un solo ciclo de reloj, garantiza que cada evento de eliminación de líneas se contabilice una única vez. Adicionalmente, se recibe como entrada el número de líneas eliminadas simultáneamente (*lines_cleared*), con un rango acotado entre 0 y 4.

El diseño interno del bloque se basa en un registro síncrono de puntuación, que almacena el valor acumulado del marcador. La actualización de dicho registro se realiza únicamente cuando se detecta un pulso de puntuación, manteniendo el valor constante el resto del tiempo. Este enfoque simplifica el control del bloque y evita incrementos indeseados de la puntuación.

Para el cálculo de los puntos obtenidos, la entidad implementa una función combinacional que asigna una cantidad fija de puntos en función del número de líneas eliminadas en una misma jugada. Esta asignación refleja la mayor recompensa asociada a la eliminación simultánea de varias líneas, incentivando estrategias de juego más complejas.

La puntuación generada se limita a un rango máximo predefinido, lo que garantiza que el valor permanezca dentro de los márgenes previstos y facilite su posterior visualización en el sistema de salida. La salida del módulo proporciona directamente el valor actual de la puntuación, que puede ser utilizado por otros bloques del sistema, como el controlador del juego o el módulo encargado de mostrarla en el display de siete segmentos.

Desde el punto de vista arquitectónico, *score_counter* constituye un bloque autónomo y fácilmente verificable, completamente desacoplado de la lógica de control y de visualización. Esta separación permite mantener un diseño modular, claro y coherente, contribuyendo a la mantenibilidad y robustez del sistema global.

3.13. Visualización en Display de Siete Segmentos(*seven_seg_driver*)

La entidad *seven_seg_driver* es la encargada de la visualización del valor de la puntuación en el display de siete segmentos integrado en la placa FPGA. Su función principal es convertir un valor entero en una representación visual decimal, gestionando de forma transparente el multiplexado de los distintos dígitos del display.

Las entradas del módulo incluyen el reloj del sistema y el valor entero que se desea mostrar, acotado en el rango de 0 a 9999. Como salidas, el bloque genera las señales correspondientes a los segmentos (*seg*) y a los ánodos (*an*) del display de siete segmentos, asumiendo una configuración de ánodo común, habitual en las placas empleadas.

Internamente, el módulo separa el valor de entrada en sus cuatro dígitos decimales mediante operaciones aritméticas simples. Esta descomposición permite mostrar el número completo utilizando un único decodificador de segmentos, simplificando la lógica del sistema.

El control del display se realiza mediante un mecanismo de multiplexado temporal, basado en un contador de refresco síncrono. Dicho contador selecciona de forma cíclica el dígito activo, activando el ánodo correspondiente y presentando en las líneas de segmentos el patrón adecuado. Gracias a la alta frecuencia de refresco, los cuatro dígitos se perciben como encendidos de forma simultánea por el ojo humano.

Para cada dígito seleccionado, el módulo emplea un decodificador combinacional BCD a siete segmentos, que traduce el valor decimal en el patrón de activación correspondiente. Los patrones generados están adaptados a la configuración de ánodo común, garantizando una correcta visualización de los números.

Desde el punto de vista del diseño, la entidad *seven_seg_driver* constituye un bloque de salida independiente, desacoplado de la lógica del juego y de la generación de la puntuación. Esta separación permite reutilizar el módulo en otros proyectos que requieran la visualización de valores numéricos y contribuye a una arquitectura clara y modular del sistema.

En conjunto, este módulo proporciona una solución eficiente y robusta para la visualización de la puntuación, completando el sistema de interacción con el usuario.

3.14. Integración del Sistema (*tetris_top*)

La entidad *tetris_top* constituye la entidad superior del sistema, siendo responsable de la integración de todos los módulos que componen el juego Tetris. Su función principal es interconectar los distintos

bloques de control, lógica de juego, visualización y entrada/salida, así como gestionar la distribución de reloj y reset, proporcionando una interfaz clara con los periféricos externos de la placa FPGA.

La entidad define como entradas el reloj principal de la placa (*clk_100MHz*), el pulsador de reset y los pulsadores utilizados para el control del juego. Como salidas, expone las señales necesarias para la visualización gráfica mediante VGA y para la visualización de la puntuación en el display de siete segmentos.

En primer lugar, *tetris_top* incorpora un bloque de generación de reloj basado en un *Clock Wizard*, encargado de generar la señal de reloj de 25 MHz necesaria para el sistema de vídeo y para el funcionamiento general del juego. El uso de este bloque permite cumplir con los requisitos temporales del estándar VGA y desacoplar el diseño del reloj principal de la FPGA. El sistema implementa además un mecanismo robusto de reset, que combina el estado de bloqueo del generador de reloj con el pulsador físico de reset. Dicho reset se sincroniza internamente con el reloj del sistema mediante una cadena de biestables, evitando problemas de metastabilidad y garantizando un arranque correcto del sistema. Adicionalmente, se distingue entre un reset global y un reset lógico del juego, que permite reiniciar una partida sin necesidad de reiniciar completamente la placa.

Las entradas procedentes de los pulsadores se procesan mediante el bloque *input_controller*, que se encarga del antirrebote y de la generación de pulsos de control sincronizados. Estos pulsos alimentan directamente al controlador de la pieza activa y, en determinadas condiciones, se reutilizan para reiniciar el juego desde los estados finales de *Game Over* o *Win*. El control global del juego se centraliza en el bloque *game_controller*, instanciado en la entidad superior. Este módulo gobierna el ciclo de juego completo, coordinando la generación de nuevas piezas, la eliminación de líneas, la progresión de niveles, la puntuación y la detección de las condiciones de victoria y derrota. A partir de su estado interno, el controlador genera señales de control que afectan al resto de bloques del sistema.

La entidad *tetris_top* integra los módulos responsables de la lógica interna del juego, incluyendo la generación pseudoaleatoria de piezas (*lfsr_random*), el controlador de la pieza activa (*current_piece_ctrl*), la memoria del tablero (*board_memory*) y el detector de colisiones (*collision_detector*). Estos bloques interactúan entre sí para garantizar un movimiento coherente de las piezas, la detección de colisiones, la fijación de piezas y la eliminación de filas completas. Adicionalmente, se implementa un mecanismo específico para la detección de condición real de *Game Over*, comprobando si una nueva pieza puede generarse en la posición inicial sin colisionar con el contenido del tablero. Esta comprobación se realiza de forma explícita en la entidad superior, reforzando la robustez del sistema.

La temporización del juego se gestiona mediante el bloque *clk_divider*, que genera señales de tick para el avance del juego y la caída automática de las piezas. Dichas señales dependen del nivel actual del juego, permitiendo aumentar progresivamente la dificultad. La puntuación se calcula mediante el bloque *score_counter*, que acumula los puntos obtenidos en función del número de líneas eliminadas simultáneamente. El valor resultante se muestra al usuario a través del bloque *seven_seg_driver*, encargado de controlar el display de siete segmentos de la placa.

La salida gráfica del juego se implementa mediante una arquitectura en dos niveles. Por un lado, el bloque *vga_controller* genera las señales de sincronismo VGA y las coordenadas del píxel actual. Por otro lado, el bloque *render_unit* utiliza dicha información junto con el estado del tablero, la pieza activa y el estado global del juego para determinar el color de cada píxel de la pantalla. Finalmente, la entidad superior adapta la señal RGB generada internamente al formato de salida requerido por la interfaz VGA de la placa.

4. Entradas y Salidas

En este apartado se describen las entradas y salidas utilizadas en el proyecto, indicando su función dentro del sistema y el papel que desempeñan en la interacción con el usuario y con los periféricos externos de la placa FPGA.

4.1. Entradas

- **Reloj del Sistema (*clk_100MHz*):** Señal de reloj principal proporcionada por la placa FPGA. A partir de esta señal se generan los relojes internos necesarios para el funcionamiento del sistema y para la salida VGA.
- **Señal de Reset (*reset_btn*):** Pulsador físico utilizado para reiniciar el sistema. Esta señal se emplea para realizar un reset controlado del diseño, garantizando un arranque correcto del juego.
- **Pulsadores de Control del Juego**
 - Botón Izquierdo (*btn_left*): Desplaza la pieza activa una posición hacia la izquierda, siempre que el movimiento sea válido.
 - Botón Derecho (*btn_right*): Desplaza la pieza activa una posición hacia la derecha, siempre que el movimiento sea válido.
 - Botón Central (*btn_rot*): Rota la pieza activa. En los estados finales (*Game Over* o *Win*), este pulsador se reutiliza para reiniciar una nueva partida sin necesidad de reiniciar la placa.
 - Botón Inferior (*btn_drop*): Acelera la caída de la pieza activa hacia abajo.

4.2. Salidas

- **Salida Gráfica VGA**
 - Señal de Sincronismo Horizontal (*vga_hsync*): Señal de sincronismo horizontal para la interfaz VGA.
 - Señal de Sincronismo Vertical (*vga_vsync*): Señal de sincronismo vertical para la interfaz VGA.
 - Canales Rojo, Verde y Azul (*vga_r*, *vga_g*, *vga_b*): Señales de color RGB, con resolución de 4 bits por canal, utilizadas para la representación gráfica del juego en pantalla.
- **Display de Siete Segmentos**
 - Señal de Control de los Segmentos (*seg*): Señales de control de los segmentos del display de siete segmentos.
 - Señales de Activación de los Ánodos (*an*): Señales de activación de los ánodos del display, utilizadas para el multiplexado de los distintos dígitos.

5. Funcionamiento del Juego

En este apartado se describe el funcionamiento general del juego Tetris, desde el inicio de una partida hasta las condiciones de finalización, detallando el comportamiento observable del sistema desde el punto de vista del usuario.

5.1. Inicio de la Partida

Al iniciar el sistema o tras reiniciar una partida, el juego comienza en el estado inicial, generándose una nueva pieza en la zona superior central del tablero. El tablero se encuentra vacío y la puntuación se inicializa a cero. El nivel de dificultad comienza en el nivel más bajo.

El sistema comprueba en este momento si la nueva pieza puede colocarse correctamente en la posición inicial. En caso contrario, se detecta inmediatamente la condición de *Game Over*.

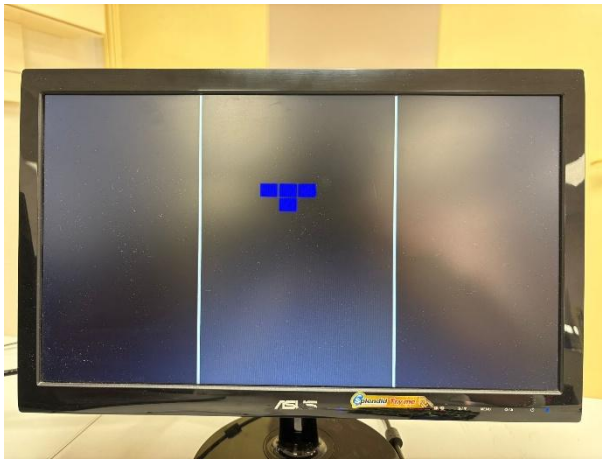


Imagen nº 1: Juego al Iniciar la Partida

5.2. Control y Movimiento de las Piezas

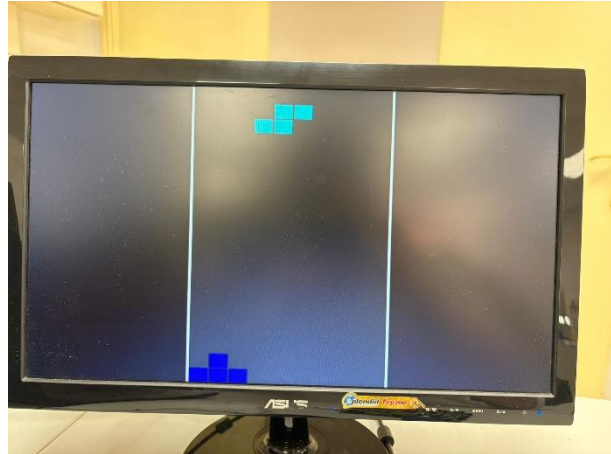
Durante el transcurso normal del juego, el usuario controla la pieza activa mediante los pulsadores de la placa:

- **Desplazamiento Horizontal a Izquierda y Derecha.**
- **Rotación de la Pieza.**
- **Caída Acelerada de la Pieza.**

Además del control manual, las piezas descienden de forma automática a intervalos regulares. Estos intervalos dependen del nivel actual del juego, de modo que, a mayor nivel, mayor velocidad de caída, incrementando progresivamente la dificultad.

Todos los movimientos están sujetos a la detección de colisiones, evitando que las piezas atraviesen los límites del tablero o se superpongan con piezas ya fijadas.

Imagen nº 2: Pieza Inicial Rotada y Desplazada



5.3. Fijación de Piezas y Eliminación de Líneas

Cuando una pieza alcanza una posición en la que ya no puede descender más, se fija de forma permanente en el tablero. A continuación, el sistema analiza el estado del tablero para detectar la presencia de filas completas.

Si una o varias filas están completamente ocupadas, estas se eliminan y las filas superiores descienden automáticamente. El número de líneas eliminadas en una misma jugada se tiene en cuenta para el cálculo de la puntuación.

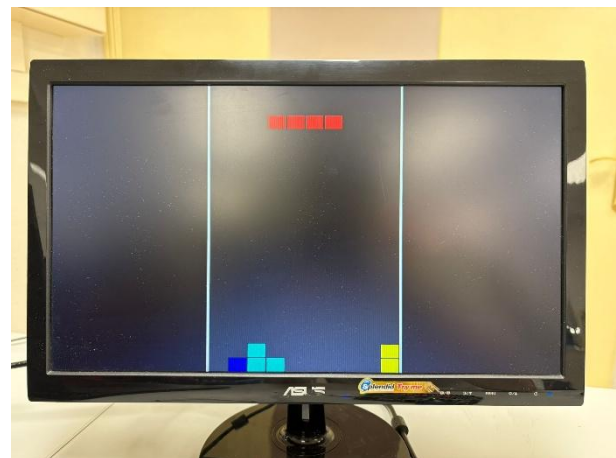
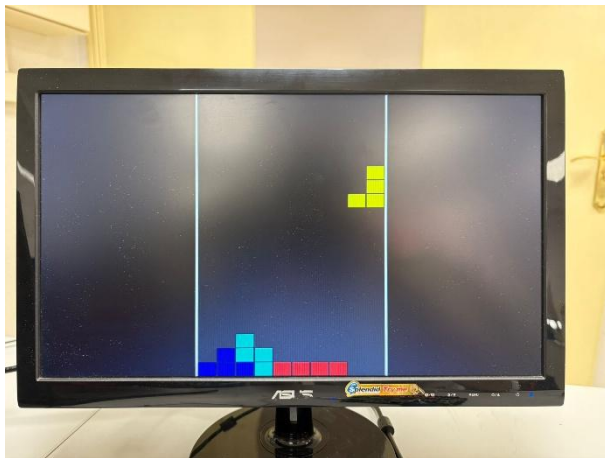


Imagen nº 3: Eliminación de una Línea Completa

5.4. Sistema de Puntuación y Niveles

La puntuación del juego aumenta en función del número de líneas eliminadas simultáneamente. El sistema recompensa la eliminación de múltiples líneas en una sola jugada con una mayor cantidad de puntos.

De manera paralela, el juego implementa un sistema de niveles, en el que el nivel aumenta tras alcanzar un determinado número de líneas eliminadas. Cada incremento de nivel conlleva un aumento de la velocidad de caída de las piezas, haciendo el juego progresivamente más exigente.

La puntuación acumulada se muestra en todo momento mediante el display de siete segmentos de la placa.

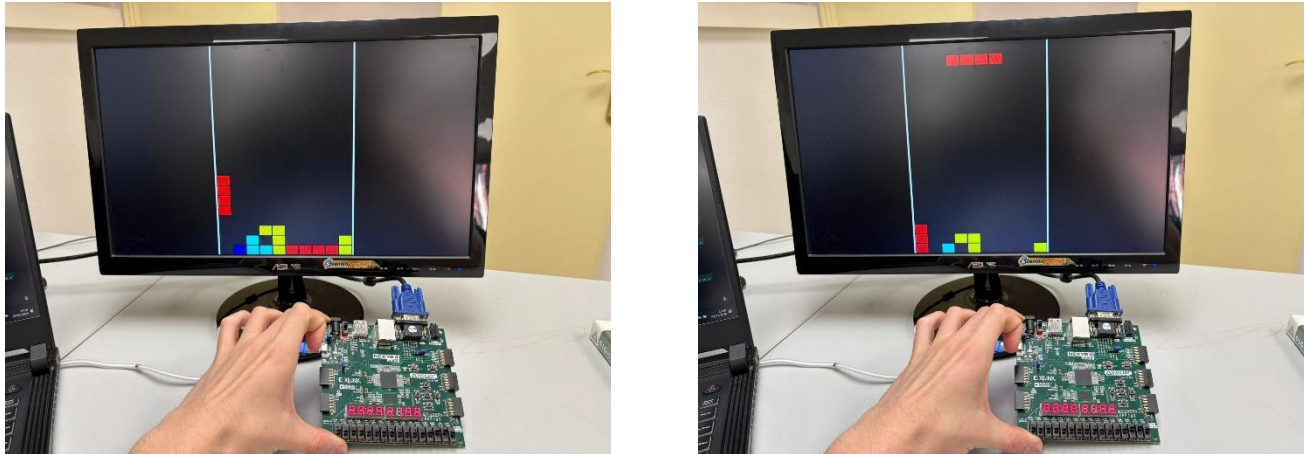


Imagen nº 4: Cambio de la Puntuación en el Display de Siete Segmentos al Eliminar una Línea

5.5. Condiciones de Victoria y Derrota

El juego puede finalizar de dos formas:

- **Derrota (Game Over):** Se produce cuando no es posible generar una nueva pieza en la posición inicial sin que colisione con piezas existentes en el tablero.
- **Victoria (Win):** Se alcanza cuando el jugador consigue superar una puntuación predefinida, indicando que se ha completado el objetivo del juego.

En ambos casos, el estado final se muestra gráficamente en la pantalla mediante mensajes diferenciados.



Imagen nº 5: Pantallas de “Game Over” y “Win”

5.6. Reinicio de la Partida

Desde los estados de *Game Over* o *Win*, el usuario puede iniciar una nueva partida sin necesidad de reiniciar la placa. Para ello, se reutiliza uno de los pulsadores de control, permitiendo reiniciar el juego de forma rápida y cómoda.

6. Testbenches

Durante el desarrollo del proyecto se emplearon testbenches en VHDL como herramienta principal de verificación en las primeras fases del diseño, especialmente para validar el funcionamiento de los bloques más críticos de forma aislada antes de su integración en el sistema completo.

En esta etapa inicial, los testbenches permitieron comprobar el comportamiento lógico de módulos como la generación de temporización, el control de entradas, la gestión de la pieza activa, la detección de colisiones y la lógica del tablero. Mediante la simulación se verificaron condiciones límite, secuencias de control y respuestas esperadas ante distintos estímulos, facilitando la detección temprana de errores de diseño.

A medida que el proyecto avanzó y los distintos bloques fueron integrándose, la verificación se complementó progresivamente con pruebas directas sobre hardware real. Tras la adquisición de una placa FPGA Basys 3 por parte del grupo, fue posible realizar pruebas continuas del sistema completo fuera del entorno de laboratorio, permitiendo validar el comportamiento global del juego en condiciones reales de funcionamiento.

En esta fase final, el proceso de depuración se basó principalmente en prueba y error sobre la FPGA, ajustando parámetros de temporización, control y visualización, y observando directamente el efecto de los cambios en la salida VGA y en la interacción con los pulsadores. Este enfoque resultó especialmente útil para detectar problemas relacionados con temporización, sincronización de señales y experiencia de usuario, difíciles de reproducir únicamente mediante simulación.

En conjunto, el uso combinado de testbenches en las fases iniciales y pruebas sobre hardware en las fases finales permitió garantizar un funcionamiento correcto y robusto del sistema, equilibrando verificación teórica mediante simulación y validación práctica en un entorno real.

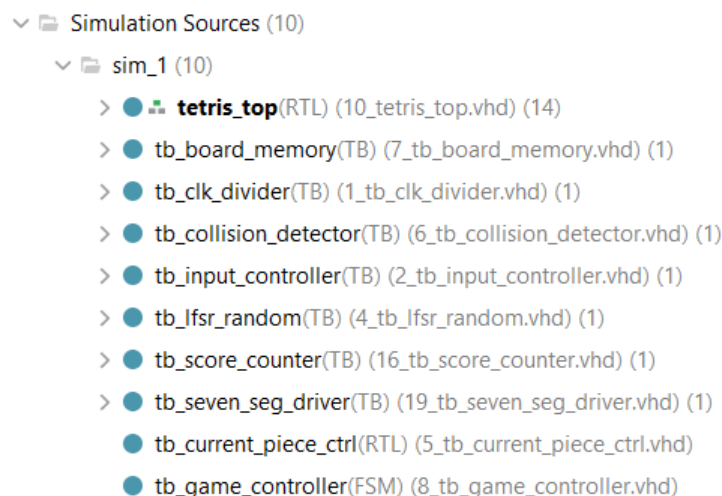


Imagen nº 6: Testbenches del Proyecto

7. Distribución del Trabajo en Grupo y Uso de GitHub

El desarrollo del proyecto se realizó de forma colaborativa, organizando el trabajo en distintas fases y áreas funcionales para facilitar la integración progresiva del sistema. Desde el inicio, el grupo acordó una arquitectura modular, lo que permitió avanzar en paralelo en el diseño, prueba e integración de los distintos bloques que componen el juego.

El repositorio Git se utilizó como herramienta central de control de versiones, permitiendo registrar la evolución del proyecto a lo largo del tiempo, integrar mejoras de forma incremental y mantener un historial claro de los cambios realizados. La actividad reflejada en el repositorio muestra un desarrollo continuado durante varias semanas, con especial intensidad durante el periodo no lectivo.

Durante las vacaciones, el grupo pudo avanzar de manera significativa gracias a la adquisición conjunta de una placa FPGA Artix-7 Basys 3, lo que permitió continuar el desarrollo y las pruebas desde casa sin depender del acceso a los laboratorios de la universidad. Este factor fue clave para acelerar la implementación, depurar el sistema y verificar su funcionamiento en hardware real.

Además del desarrollo de código, el trabajo en grupo incluyó tareas de pruebas funcionales, validación del comportamiento del juego, ajuste de parámetros y elaboración de la documentación, realizadas de forma coordinada para garantizar la coherencia del diseño final.

En conjunto, el uso del repositorio Git y la organización del trabajo en grupo han permitido un desarrollo estructurado y progresivo del proyecto, facilitando la integración de los distintos módulos y la estabilidad del sistema final.

Enlace al GitHub del Trabajo: https://github.com/Felipearri21/SED_Tetris



Imagen nº 7: Prueba de la Participación de Todos los Integrantes del Grupo

8. Funcionalidades Implementadas

A continuación, se enumeran las principales funcionalidades implementadas en el proyecto, describiendo de forma breve el comportamiento aportado por cada una de ellas.

- **Visualización Gráfica mediante Salida VGA:** El juego se representa gráficamente en una pantalla VGA, mostrando el tablero, las piezas en movimiento y los estados finales del juego, utilizando una resolución estándar compatible con las placas FPGA empleadas.
- **Control del Juego mediante Pulsadores Físicos:** El usuario interactúa con el juego exclusivamente a través de los pulsadores integrados en la placa, permitiendo el desplazamiento horizontal, la rotación de las piezas y la caída acelerada.
- **Sistema de Antirrebote y Generación de Pulsos:** Las entradas de los pulsadores incorporan un sistema de antirrebote y generación de pulsos de un solo ciclo, garantizando un control preciso y fiable de las acciones del usuario.
- **Movimiento Automático de las Piezas:** Las piezas descienden de forma automática a intervalos regulares, simulando la gravedad del juego, sin necesidad de intervención del usuario.
- **Incremento Progresivo de la Dificultad:** La velocidad de caída de las piezas aumenta conforme se incrementa el nivel del juego, introduciendo una dificultad progresiva.
- **Detección de Colisiones:** El sistema detecta colisiones entre la pieza activa, los límites del tablero y las piezas ya fijadas, impidiendo movimientos inválidos.
- **Fijación de Piezas en el Tablero:** Cuando una pieza ya no puede descender, se fija permanentemente en el tablero, pasando a formar parte del estado del juego.
- **Eliminación Automática de Líneas Completas:** El sistema detecta filas completamente ocupadas y las elimina automáticamente, reorganizando el tablero de acuerdo con las reglas del juego.
- **Sistema de Puntuación Dinámico:** Se implementa un sistema de puntuación que asigna puntos en función del número de líneas eliminadas simultáneamente, incentivando jugadas más complejas.
- **Visualización de la Puntuación en Display de Siete Segmentos:** La puntuación acumulada se muestra en tiempo real mediante el display de siete segmentos integrado en la placa.
- **Generación Pseudoaleatoria de Piezas:** Las nuevas piezas se generan de forma pseudoaleatoria, garantizando variedad en el desarrollo de la partida.
- **Gestión del Estado del Juego mediante Máquina de Estados:** El flujo del juego se controla mediante una máquina de estados que coordina la generación de piezas, la eliminación de líneas y los estados finales.

- **Detección de Condición de Derrota (*Game Over*):** El sistema detecta correctamente la imposibilidad de generar nuevas piezas en la posición inicial, finalizando la partida cuando corresponde.
- **Condición de Victoria (*Win*) basada en Puntuación:** Se implementa una condición de victoria al alcanzar una puntuación predefinida, mostrando un estado final diferenciado.
- **Reinicio de Partida sin Reset de la Placa:** Desde los estados finales, el usuario puede reiniciar una nueva partida utilizando un pulsador, sin necesidad de reiniciar completamente el sistema.

9. Problemas y Soluciones

9.1. Problema de Conexión y Visualización en Pantalla por VGA

Durante las primeras fases del desarrollo, la salida VGA no mostraba una imagen correcta o estable, observándose en ocasiones pantalla en negro, desincronización o ausencia total de señal visible. Este tipo de comportamiento es habitual cuando la temporización VGA no se ajusta exactamente al estándar esperado por el monitor, o cuando el reloj de píxel no tiene la frecuencia adecuada. En particular, el uso de divisores de reloj no dedicados o parámetros incorrectos de sincronismo provocaba que la señal no fuese interpretada correctamente.

El problema se resolvió implementando un controlador VGA basado en el estándar 640×480 a 60 Hz, utilizando parámetros de sincronismo VESA correctos. Asimismo, se incorporó un bloque *Clock Wizard* de Vivado para generar un reloj de píxel estable de 25 MHz, garantizando una temporización precisa y libre de inestabilidades. Adicionalmente, se empleó la señal *video_active* para limitar la generación de color a la región visible, evitando artefactos fuera de la zona activa.

9.2. Problema de Colisiones y Coherencia Temporal por Frecuencias de Reloj

En versiones iniciales del diseño se detectaron comportamientos inconsistentes en el movimiento de las piezas, como desplazamientos que en ocasiones se permitían y en otras no, o colisiones detectadas de forma aparentemente errónea. Este problema estaba relacionado con una falta de sincronización clara entre la lógica de movimiento, la detección de colisiones y la temporización del sistema, provocada por el uso directo del reloj principal sin una gestión explícita de los tiempos de juego.

Para solucionar este problema se introdujo una separación clara de la temporización del juego, implementando un divisor de reloj dedicado (*clk_divider*) que genera señales de tick bien definidas. El movimiento automático y manual de las piezas pasó a depender exclusivamente de estos eventos temporizados, mientras que la validación de movimientos se centralizó mediante señales de habilitación (*can_move_**) generadas por el detector de colisiones. De este modo se garantizó un comportamiento coherente y determinista del sistema.

9.3. Problema de Reset y Arranque No Determinista del Sistema

Se observaron problemas durante el arranque del sistema, como estados iniciales incorrectos o comportamientos no reproducibles tras la programación de la FPGA. Además, al trabajar con distintas placas, la polaridad del pulsador de reset no era siempre la misma. Estos problemas son frecuentes cuando el reset no se sincroniza correctamente con el reloj del sistema o cuando el diseño comienza a funcionar antes de que el reloj generado esté estabilizado.

Se implementó un sistema de reset robusto, combinando el estado del pulsador físico con la señal *locked* del generador de reloj. El reset interno se sincronizó mediante una cadena de biestables, evitando problemas de metastabilidad. Asimismo, se parametrizó la polaridad del pulsador para adaptarse a distintas placas, garantizando un arranque consistente y fiable del sistema en todos los casos.

9.4. Problema de Reinicio de Partida sin Reset Físico

Inicialmente, al intentar reiniciar una partida desde los estados finales (*Game Over* o *Win*) sin reiniciar la placa, se producían incoherencias en el estado del juego, como puntuación incorrecta, tablero no limpio o piezas en estados no válidos. Esto se debía a que solo algunos bloques se reiniciaban, mientras que otros conservaban información de la partida anterior.

La solución consistió en introducir un reset lógico de juego, independiente del reset global de la FPGA. Este reset es generado por el controlador del juego y se aplica únicamente a los bloques que almacenan el estado de la partida. De esta forma se consigue reiniciar completamente el juego sin necesidad de reconfigurar la FPGA, manteniendo un comportamiento coherente y controlado.

9.5. Problema en la Gestión de la Puntuación y Borrado de Líneas

Durante el desarrollo se detectaron posibles inconsistencias en la actualización de la puntuación, especialmente relacionadas con el borrado de varias líneas simultáneamente. En sistemas de este tipo, un diseño inadecuado puede provocar incrementos múltiples de puntuación para un mismo evento o el uso de valores inestables del número de líneas borradas.

Para evitar este problema, se sincronizó la actualización de la puntuación mediante un pulso de un solo ciclo (*score_pulse*) generado únicamente en el estado correspondiente de la máquina de estados del juego. Además, el número de líneas eliminadas se almacenó en un registro dedicado dentro de la memoria del tablero, asegurando que el valor utilizado para el cálculo de la puntuación fuese estable y coherente.

9.6. Problema de Bloqueo de la Pieza al Tocar el Suelo

En algunas versiones iniciales, al alcanzar el fondo del tablero una pieza podía seguir respondiendo a movimientos laterales o rotaciones durante un breve intervalo, generando una sensación de comportamiento incorrecto y alejándose del funcionamiento clásico del Tetris.

Se incorporó una lógica explícita de congelación de la pieza activa. En el momento en que la pieza deja de poder descender, se emite una solicitud de fijación y se bloquea cualquier movimiento adicional

hasta que el controlador del juego genera una nueva pieza. Esta solución garantiza una transición clara entre caída, fijación y generación de la siguiente pieza.

9.7. Problemas de Integración y Organización del Proyecto en Vivado

Durante la reorganización del proyecto y la eliminación o modificación de ciertos ficheros, aparecieron errores de síntesis e implementación debidos a instancias de entidades inexistentes o desajustes entre los puertos definidos y los conectados. Este tipo de errores es habitual en proyectos modulares de tamaño medio.

Se revisó la jerarquía completa del diseño, reconstruyendo la entidad superior y verificando la coherencia entre todas las instancias. Además, se utilizó el *schematic* generado por Vivado como herramienta de comprobación, asegurando que todas las conexiones entre bloques eran correctas y que el proyecto reflejaba fielmente la arquitectura prevista.

10. Conclusiones

El proyecto desarrollado ha permitido implementar con éxito un juego Tetris completo en VHDL, ejecutable sobre una plataforma FPGA y haciendo uso exclusivo de los recursos hardware disponibles en la placa. A lo largo del trabajo se han abordado tanto aspectos de diseño digital como de integración de sistemas, cumpliendo los objetivos planteados para la asignatura.

La arquitectura adoptada, basada en una estructura modular y jerarquizada, ha facilitado el desarrollo, la depuración y la integración del sistema. La separación clara entre bloques de control, lógica de juego, gestión del tablero, temporización y visualización ha resultado clave para mantener un diseño comprensible, escalable y robusto.

Desde el punto de vista funcional, el sistema reproduce fielmente el comportamiento clásico del juego Tetris, incorporando control mediante pulsadores, detección de colisiones, eliminación de líneas, sistema de puntuación y gestión de niveles, así como estados finales diferenciados de victoria y derrota. La visualización gráfica mediante salida VGA y la presentación de la puntuación en el display de siete segmentos proporcionan una interacción completa con el usuario.

El proyecto ha permitido aplicar de forma práctica conceptos fundamentales de la asignatura, como el uso de máquinas de estados, temporización mediante divisores de reloj, manejo de señales síncronas y combinacionales, así como la integración de periféricos externos. Asimismo, el uso de herramientas profesionales como Vivado y Git ha reforzado la comprensión del flujo de desarrollo de sistemas digitales complejos.

En conjunto, el trabajo realizado demuestra la viabilidad de implementar sistemas interactivos de cierta complejidad sobre FPGA utilizando VHDL, y ha supuesto una experiencia formativa valiosa tanto a nivel técnico como organizativo, consolidando los conocimientos adquiridos a lo largo de la asignatura.